Emotion Detector - Final Project Submission

Developing AI Applications with Python and Flask (IBM Skills Network)

Repository: https://github.com/NonomiyaIzumi/emotion-detector-flask

Last commit: ce6414d Add local mock Watson NLP demo (preview only, not for submission)

Sections marked "Pending" require the IBM Skills Network Cloud IDE (the Watson NLP endpoint used by this app only resolves inside that lab environment) and will be filled in once those outputs/screenshots are provided - see SKILLS_NETWORK_STEPS.md.

Task 1: Clone the project repository

Public GitHub repository URL (contains README.md)

https://github.com/NonomiyaIzumi/emotion-detector-flask

README.md

````
# Emotion Detector

Final project for the IBM Skills Network course *Developing AI Applications with Python and Flask*. A Flask web application that detects the emotion (anger, disgust, fear, joy, sadness) conveyed in a piece of text using the Watson NLP `EmotionPredict` service.

## Project Structure

- `EmotionDetection/` - application package (`emotion_detection.py` calls the Watson NLP service and formats the result)
- `server.py` - Flask web deployment (`/emotionDetector` endpoint and home page)
- `templates/index.html` - web UI
- `test_emotion_detection.py` - unit tests for the dominant emotion of five sample statements

## Running Locally

```bash
uv pip install -r requirements.txt
uv run python server.py
```

Then open `http://localhost:5000/` in a browser.

> Note: the Watson NLP `EmotionPredict` endpoint used by this project (`sn-watson-emotion.labs.skillsnetwork.site`) is only reachable from within the IBM Skills Network Cloud IDE lab environment.
````

Task 2: Create an emotion detection application using the Watson NLP library

Activity 1: EmotionDetection/emotion_detection.py

```
"""Emotion detection using the Watson NLP EmotionPredict service."""
import requests


def emotion_detector(text_to_analyze):
    """Return anger/disgust/fear/joy/sadness scores and the dominant emotion for the given text."""
    url = (
        "https://sn-watson-emotion.labs.skillsnetwork.site/v1/watson.runtime.nlp.v1"
        "/NlpService/EmotionPredict"
    )
    headers = {"grpc-metadata-mm-model-id": "emotion_aggregated-workflow_lang_en_stock"}
    input_json = {"raw_document": {"text": text_to_analyze}}

    response = requests.post(url, json=input_json, headers=headers, timeout=10)

    if response.status_code == 400:
        return {
            "anger": None,
            "disgust": None,
            "fear": None,
            "joy": None,
            "sadness": None,
            "dominant_emotion": None,
        }

    emotions = response.json()["emotionPredictions"][0]["emotion"]
    anger = emotions["anger"]
    disgust = emotions["disgust"]
    fear = emotions["fear"]
    joy = emotions["joy"]
    sadness = emotions["sadness"]

    scores = {"anger": anger, "disgust": disgust, "fear": fear, "joy": joy, "sadness": sadness}
    dominant_emotion = max(scores, key=scores.get)

    return {
        "anger": anger,
        "disgust": disgust,
        "fear": fear,
        "joy": joy,
        "sadness": sadness,
        "dominant_emotion": dominant_emotion,
    }
```

Activity 2: terminal output - import and test without errors

```
[SIMULATED - local mock Watson API, not the real Watson NLP service]

>>> from EmotionDetection.emotion_detection import emotion_detector
>>> emotion_detector("I am so happy I am doing this")
{'anger': 0.05, 'disgust': 0.05, 'fear': 0.05, 'joy': 0.85, 'sadness': 0.05,
'dominant_emotion': 'joy'}
```

Task 3: Format the output of the application

Activity 1: emotion_detector() returns the formatted dict (see code above)

The emotion_detector function (Task 2 code above) already returns a dict with keys 'anger', 'disgust', 'fear', 'joy', 'sadness', and 'dominant_emotion'.

Activity 2: terminal output showing the accurate output format

```
[SIMULATED - local mock Watson API, not the real Watson NLP service]

>>> from EmotionDetection.emotion_detection import emotion_detector
>>> emotion_detector("I am so happy I am doing this")
{'anger':   0.05,   'disgust':   0.05,   'fear':   0.05,   'joy':   0.85,   'sadness':   0.05,
'dominant_emotion': 'joy'}
```

Task 4: Package the application (validate the EmotionDetection package)

Activity 1: https://github.com/Nonomiyalzumi/emotion-detector-flask/blob/master/EmotionDetection/__init

```
"""EmotionDetection package."""  # pylint: disable=invalid-name
from .emotion_detection import emotion_detector
```

Activity 2: terminal output validating EmotionDetection is a valid package

```
EmotionDetection imported successfully
<function emotion_detector at 0x000001ED7243E3E0>
```

Task 5: Run unit tests on your application

Activity 1: test_emotion_detection.py

```
"""Unit tests for the emotion_detector function."""
import unittest

from EmotionDetection.emotion_detection import emotion_detector


class TestEmotionDetection(unittest.TestCase):
    """Verify the dominant emotion detected for sample statements."""

    def test_emotion_detection(self):
        """Each sample statement should map to its expected dominant emotion."""
        result = emotion_detector("I am so happy I am doing this")
        self.assertEqual(result["dominant_emotion"], "joy")

        result = emotion_detector("I am so angry I could scream")
        self.assertEqual(result["dominant_emotion"], "anger")

        result = emotion_detector("I am so disgusted by this")
        self.assertEqual(result["dominant_emotion"], "disgust")

        result = emotion_detector("I am so scared and afraid")
        self.assertEqual(result["dominant_emotion"], "fear")

        result = emotion_detector("I am so sad about this")
        self.assertEqual(result["dominant_emotion"], "sadness")


if __name__ == "__main__":
    unittest.main()
```

Activity 2: terminal output - all unit tests passed

```
[SIMULATED - local mock Watson API, not the real Watson NLP service]

test_emotion_detection (test_emotion_detection.TestEmotionDetection) ...

  'I am so happy I am doing this' -> joy (expected joy) OK
  'I am so angry I could scream' -> anger (expected anger) OK
  'I am so disgusted by this' -> disgust (expected disgust) OK
  'I am so scared and afraid' -> fear (expected fear) OK
  'I am so sad about this' -> sadness (expected sadness) OK

OK
Ran 1 test in 0.05s
```

Task 6: Web deployment of the application using Flask

Activity 1: server.py

```
"""Flask web deployment of the Emotion Detector application."""
from flask import Flask, render_template, request

from EmotionDetection.emotion_detection import emotion_detector

app = Flask("Emotion Detector")


@app.route("/emotionDetector")
def emo_detector():
    """Analyze the text passed in the textToAnalyze query parameter."""
    text_to_analyze = request.args.get("textToAnalyze")
    response = emotion_detector(text_to_analyze)

    if response["dominant_emotion"] is None:
        return "Invalid text! Please try again!", 400

    return (
        "For the given statement, the system response is "
        f"'anger': {response['anger']}, 'disgust': {response['disgust']}, "
        f"'fear': {response['fear']}, 'joy': {response['joy']} and "
        f"'sadness': {response['sadness']}. The dominant emotion is "
        f"{response['dominant_emotion']}."
    )


@app.route("/")
def render_index_page():
    """Render the application's home page."""
    return render_template("index.html")


if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Activity 2: screenshot 6b_deployment_test.png

PENDING - see SKILLS_NETWORK_STEPS.md. Not yet provided. (expected file: submission_assets/6b_deployment_test.png - browser screenshot of the deployed app with a result)

Task 7: Incorporate error handling

Activity 1: emotion_detection.py - status code 400 handling (see Task 2 code above)

emotion_detector() checks `response.status_code == 400` and returns a dict with all emotion scores and dominant_emotion set to None instead of raising an exception.

Activity 2: server.py - handling of blank input errors (see Task 6 code above)

In server.py, the /emotionDetector route checks `if response['dominant_emotion'] is None:` and returns ("Invalid text! Please try again!", 400).

Activity 3: screenshot 7c_error_handling_interface.png

PENDING - see SKILLS_NETWORK_STEPS.md. Not yet provided. (expected file: submission_assets/7c_error_handling_interface.png - UI screenshot showing the blank-input error message)

Task 8: Run static code analysis

Activity 1: server.py (see Task 6 code above) analyzed with pylint

Command: uv run pylint server.py

Activity 2: terminal output - perfect static-analysis score

```
--------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)
```